

Neural Networks Essentials

You Only Look Once (YOLO)

Denis C. Ilie-Ablachim

March 18, 2026

Course Overview

Introduction to Object Detection

YOLO Architecture

Bounding Box Parameterization and Prediction

Loss Function and Training Details

Mathematical Insights and Theoretical Considerations

Practical Implementation Insight

Real-World Applications and Extensions

Summary and Key Takeaways

Q&A and Discussion

Object Detection Overview

- ▶ **Traditional Computer Vision Approaches:**
 - ▶ Sliding Window + Classifier (e.g., SVM, HOG features).
 - ▶ Region-based proposals (e.g., R-CNN, Fast R-CNN).
- ▶ **Deep Learning for Object Detection:**
 - ▶ Two-stage detectors: R-CNN family (Region Proposal + Classification).
 - ▶ One-stage detectors: YOLO, SSD, RetinaNet.
- ▶ **Key Goal:** Predict **bounding boxes** and **class probabilities** for multiple objects in an image.

Motivation for YOLO

- ▶ Single forward pass for detection (hence *You Only Look Once*).
- ▶ **Real-time** performance: Significant for applications like autonomous driving, robotics, video surveillance.
- ▶ Unified approach simplifies the pipeline: direct bounding-box regression and class probability prediction.

High-Level Architecture

Steps in YOLO (original version):

1. Input image is divided into an $S \times S$ grid.
2. Each grid cell is responsible for detecting objects whose center falls into it.
3. Each cell predicts:
 - ▶ One or more bounding boxes (with confidence scores).
 - ▶ Class probability distribution.

Backbone Network:

- ▶ CNN-based feature extraction.
- ▶ Original YOLO: 24 convolutional layers + 2 fully connected layers.
- ▶ Variations: YOLOv2, YOLOv3, YOLOv4, YOLOv5, etc., each with architectural improvements.

Grid-Based Detection

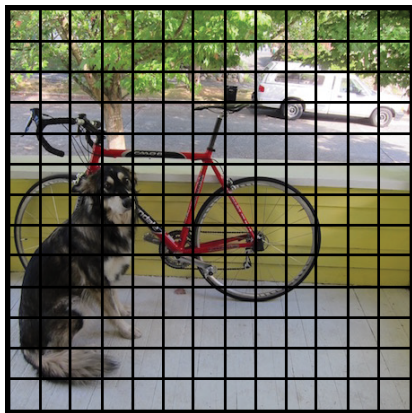


Figure: An illustration of splitting an image into an $S \times S$ grid.

- If the center of an object (e.g., a dog) lies in cell (i, j) , that cell is responsible for predicting the bounding box and class for the dog.

Bounding Box Parameterization

- ▶ Each cell predicts B bounding boxes.
- ▶ Each bounding box has 5 predictions:

$$(x, y, w, h, \hat{p})$$

where:

- ▶ (x, y) : center coordinates *relative* to the cell location.
- ▶ (w, h) : width and height of the bounding box (often parameterized via transformations, e.g. exponentiation to keep dimensions positive).
- ▶ $\hat{p}$: confidence score, defined as:

$$\hat{p} = P(\text{object present}) \times \text{IOU}(\text{pred}, \text{truth}).$$

- ▶ In addition, each cell predicts a probability distribution over C classes, e.g. $(p_1, p_2, \dots, p_C)$.

Bounding Box Equations (Typical Form)

- ▶ Let the coordinates of the top-left corner of cell (i, j) be (C_x, C_y) and the width, height of the cell be (C_w, C_h) .
- ▶ YOLO often parameterizes:

$$b_x = \sigma(t_x) + C_x, \quad b_y = \sigma(t_y) + C_y,$$

$$b_w = p_w e^{t_w}, \quad b_h = p_h e^{t_h},$$

where (t_x, t_y, t_w, t_h) are predicted by the network, $\sigma(\cdot)$ is the sigmoid function, and (p_w, p_h) might be prior anchor dimensions (in advanced YOLO versions).

- ▶ Confidence score $\hat{p}$ is also typically passed through a sigmoid.

Loss Function Components

- ▶ YOLO's loss function balances:
 1. **Localization loss:** Squared error between predicted box and ground truth box.
 2. **Confidence loss:** Measures whether an object is in the predicted box and how well it fits (IOU).
 3. **Classification loss:** Cross-entropy or MSE for class probabilities.
- ▶ Weighted sum of these terms. For example:

$$\mathcal{L} = \lambda_{\text{coord}} \sum_{\text{obj}} (x - \hat{x})^2 + (y - \hat{y})^2 + \dots + \lambda_{\text{noobj}} \sum_{\text{no obj}} (0 - \hat{p})^2 + \dots$$

- ▶ λ_{coord} and λ_{noobj} are hyperparameters to balance localization vs. confidence loss.

Training Pipeline

1. **Dataset Preparation:** Typical datasets: PASCAL VOC, COCO. Need bounding box labels + object classes.
2. **Model Initialization:** Often pretrained on ImageNet for classification, then finetuned for detection.
3. **Forward Pass:** Compute predictions for each cell and bounding box.
4. **Loss Computation:** Compare predicted bounding boxes and class probabilities with ground truth.
5. **Backpropagation:** Update weights using an optimizer (e.g., SGD or Adam).
6. **Post-processing:** Non-Maximum Suppression (NMS) to remove duplicate predictions of the same object.

Intersection over Union (IOU)

- ▶ **IOU** between two bounding boxes A and B :

$$\text{IOU}(A, B) = \frac{\text{Area}(A \cap B)}{\text{Area}(A \cup B)}.$$

- ▶ Key metric for:
 - ▶ Evaluating detection quality during training.
 - ▶ Computing the confidence score $\hat{p}$ in YOLO.
 - ▶ Applying NMS to filter bounding boxes.

Confidence Score and Class Probability

- ▶ The predicted confidence score
 $\hat{p} \approx P(\text{object}) \times \text{IOU}(\text{pred}, \text{truth})$.
- ▶ If no object is present, then $P(\text{object}) = 0$ ideally.
- ▶ **Class Probability:** Each bounding box also has an associated probability distribution p_c over classes:

$$p_c = P(\text{class } c \mid \text{object detected}).$$

- ▶ Final detection confidence for class c is often $\hat{p} \times p_c$.

High-Level Pseudocode

YOLO-like Model Forward Pass (Simplified):

$$\text{output} = \text{ConvNet}(x_{\text{input}})$$

- ▶ Suppose the output is a $S \times S \times (B \times 5 + C)$ tensor.
- ▶ **Interpretation:**
 - ▶ For each cell: B bounding boxes $\times$ 5 parameters $(x, y, w, h, \hat{p})$.
 - ▶ Plus C class probabilities.

During training:

1. Compute $\mathcal{L}$ (localization, confidence, classification).
2. Backprop through the network.
3. Update weights using optimizer (SGD, Adam, etc.).

During inference:

1. Parse the output grid predictions.
2. Filter low-confidence boxes.
3. Apply Non-Maximum Suppression (NMS) to remove duplicates.

Implementation Tips and Tricks

▶ **Anchor Boxes:**

- ▶ Predefined "anchor" (or "prior") boxes of different scales/aspect ratios.
- ▶ Helps capture multi-scale objects more effectively (e.g. YOLOv2+).

▶ **Training Strategy:**

- ▶ Start with lower resolution for faster iteration.
- ▶ Use data augmentation (random cropping, flipping, color jitter).
- ▶ Keep track of bounding box regression stability.

▶ **Batch Normalization:**

- ▶ Often employed in YOLO layers to stabilize training and speed convergence.

▶ **Learning Rate Scheduling:**

- ▶ Warm-up phases, step decay, or cosine annealing commonly used.

Applications

- ▶ **Autonomous Driving:** Real-time pedestrian/vehicle detection.
- ▶ **Surveillance:** Identify suspicious objects or persons quickly.
- ▶ **Robotics:** On-the-fly object detection for grasping or navigation.
- ▶ **Medical Imaging:** Localize tumors or anomalies in real-time if specialized hardware permits.

Extensions and Improvements

- ▶ **YOLOv2/YOLO9000:** Improved anchor box mechanism, multi-scale training.
- ▶ **YOLOv3:** Introduced multi-scale feature detection (feature pyramid) and improved accuracy.
- ▶ **Subsequent Versions (v4, v5, v7, v8):** Additional optimizations like CSPDarknet, more efficient training routines, improved neck/backbone designs, etc.
- ▶ **Tiny YOLO:** Lightweight version for embedded systems.

Key Takeaways

- ▶ **Unified Framework:** YOLO treats detection as a regression problem, simplifying the pipeline.
- ▶ **Speed vs. Accuracy:** YOLO is extremely fast compared to two-stage methods, though it may have slightly lower accuracy for small objects (improved in newer versions).
- ▶ **Important Concepts:**
 - ▶ Grid-based detection.
 - ▶ Bounding box parameterization.
 - ▶ Confidence scores and class probabilities.
 - ▶ Multi-scale training and anchor boxes.
- ▶ **Loss Function Design:** Carefully balances localization, confidence, and classification error.

Further Reading

- ▶ **Original YOLO paper:**
You Only Look Once: Unified, Real-Time Object Detection
Redmon & Farhadi, CVPR 2016.
- ▶ **YOLOv2/YOLO9000 paper:**
YOLO9000: Better, Faster, Stronger
Redmon & Farhadi, CVPR 2017.
- ▶ **YOLOv3 paper:**
YOLOv3: An Incremental Improvement
Redmon & Farhadi, 2018.
- ▶ **Associated GitHub Repositories:**
Look at implementations (Darknet, PyTorch, TensorFlow).

Discussion

- ▶ Practical tips for better performance?
- ▶ How does YOLO compare to other one-stage detectors?
- ▶ Can YOLO be adapted for semantic/instance segmentation?
- ▶ Challenges in detecting small or occluded objects?

Conclusion

Main Points:

- ▶ YOLO is a landmark architecture for real-time object detection.
- ▶ A single forward pass helps it achieve excellent speed.
- ▶ Continuous evolution (v2, v3, v4, ...) improves accuracy and efficiency.
- ▶ Understanding loss function and bounding box parameterization is key for customization.